

IntellMeet Backend Development Log – Phase 1 (Authentication & Authorization)

Project

IntellMeet – AI Powered Enterprise Meeting & Collaboration Platform

Phase Completed (By: Rajyavardhan Radhey)

Authentication & Authorization Module (Priority 1)

Objective

The objective of this phase was to establish a secure and scalable authentication system that would serve as the foundation for all future modules such as Teams, Meetings, Tasks, Chat, Notifications, and AI features.

The authentication layer was designed using industry-standard security practices and a service-based architecture to ensure maintainability, scalability, and security.

Technologies Used

Backend Framework

- Node.js
- Express.js

Database

- MongoDB
- Mongoose

Authentication

- JWT (JSON Web Token)
- Refresh Tokens
- bcryptjs

Validation

- Zod

Security

- Helmet
- CORS
- Express Rate Limiter

Development Tools

- Nodemon
 - Thunder Client
 - MongoDB Compass
-

Architecture Implemented

The authentication module follows a layered architecture:

Client

- Routes
- Validation Middleware
- Authentication Middleware
- Controller
- Service
- Model
- MongoDB

Responsibilities:

Routes:

- Define API endpoints.

Controllers:

- Handle requests and responses.

Services:

- Contain business logic.

Models:

- Define MongoDB schema structure.

Middlewares:

- Validation
 - Authentication
 - Authorization
-

Database Schema Created

Collection:

Users

Fields:

- firstName
- lastName
- email
- password
- avatar
- role
- isVerified
- refreshToken
- lastLogin
- createdAt
- updatedAt

Security Enhancements:

password:

- Hidden using select: false

refreshToken:

- Hidden using select: false

APIs Implemented

Signup

POST /api/auth/signup

Purpose:

Register a new user.

Features:

- Email uniqueness check
 - Password hashing
 - Validation using Zod
-

Login

POST /api/auth/login

Purpose:

Authenticate user.

Features:

- Password verification
 - Access Token generation
 - Refresh Token generation
 - Login timestamp update
-

Get Current User

GET /api/auth/me

Purpose:

Return currently authenticated user.

Features:

- JWT verification
 - Protected route
-

Refresh Token

POST /api/auth/refresh-token

Purpose:

Generate new access token.

Features:

- Refresh token validation
 - Access token regeneration
-

Logout

POST /api/auth/logout

Purpose:

Invalidate refresh token.

Features:

- Refresh token removal from database

Security Features Implemented

Password Hashing

Library:

bcryptjs

Implementation:

- Passwords are never stored in plain text.
- Salt rounds = 12

Benefits:

- Protects against database leaks.
- Makes password cracking significantly harder.

JWT Authentication

Access Token:

Contains:

- User ID
- Role

Purpose:

Used for protected API requests.

Expiry:

15 minutes

Benefits:

- Stateless authentication
- High performance

Refresh Token Strategy

Purpose:

Allows generating new access tokens without forcing users to login again.

Benefits:

- Improved user experience
- Reduced authentication requests

Protected Routes

Middleware:
authenticate

Purpose:

Verifies:

- Authorization header exists
- Token format is correct
- JWT signature is valid
- User exists in database

RBAC (Role Based Access Control)

Roles Implemented:

- MEMBER
- MANAGER
- ADMIN
- SUPER_ADMIN

Authorization Middleware:

authorize(...roles)

Purpose:

Restrict access based on user role.

Example:

Only ADMIN and SUPER_ADMIN can access certain routes.

Validation Layer

Library:

Zod

Purpose:

Validate incoming request data before it reaches controllers.

Benefits:

- Prevents invalid data from entering database
 - Reduces runtime errors
 - Improves API reliability
-

Folder Structure Used

src/

config/

models/

routes/

controllers/

services/

middlewares/

validators/

utils/

constants/

Benefits:

- Separation of concerns
 - Easy maintenance
 - Scalable architecture
-

Challenges Faced

1. MongoDB Connection Setup

Issue:

MongoDB was not initially installed locally.

Solution:

- Installed MongoDB Community Server
 - Configured local connection
 - Verified using MongoDB Compass
-

2. Missing Dependencies

Issue:

Packages such as dotenv and nodemon were missing.

Solution:

Installed required dependencies using npm.

3. express-mongo-sanitize Compatibility Issue

Issue:

Application crashed with:

Cannot set property query of IncomingMessage

Reason:

Package compatibility issues with current Node.js version.

Solution:

Removed express-mongo-sanitize and relied on:

- Zod Validation
 - Mongoose Validation
-

4. JWT Secret Misconfiguration

Issue:

Refresh Token API returned:

Invalid refresh token

Reason:

JWT_ACCESS_SECRET and JWT_REFRESH_SECRET were changed after token generation.

Solution:

- Updated secrets
 - Generated fresh tokens
 - Re-tested APIs
-

5. Authorization Header Errors

Issue:

Protected routes returned:

Unauthorized

Invalid token

Reason:

Incorrect header formatting.

Solution:

Used:

Authorization: Bearer <access_token>

6. Sensitive Data Exposure

Issue:

Password and refresh token were being returned in API responses.

Solution:

- Added select: false
 - Sanitized controller responses
-

Outcome

At the end of Phase 1, the application has a secure authentication and authorization foundation.

Implemented Features:

- ✓ User Registration
- ✓ User Login
- ✓ Password Hashing
- ✓ JWT Authentication
- ✓ Refresh Token System
- ✓ Logout
- ✓ Protected Routes
- ✓ Validation Middleware
- ✓ RBAC
- ✓ Service Layer Architecture

This authentication system is production-ready and can now support future modules such as Teams, Meetings, Chat, Tasks, AI Summaries, Notifications, and Analytics.

Phase 2 — Team Workspace Module

Objective

Create a Team Management System similar to:

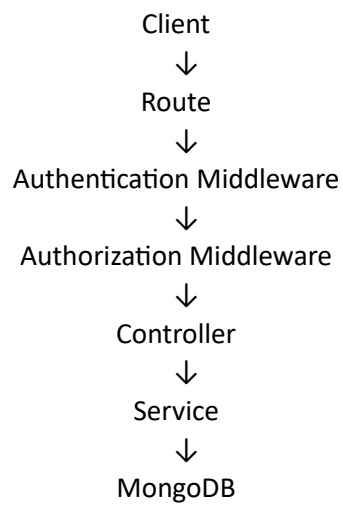
Microsoft Teams

Slack Workspaces

Google Workspace Teams

Every meeting, task, chat, board, and AI summary will belong to a Team.

Architecture



Team Schema

Collection:

teams

Fields:

```
{
  name,
  description,
  logo,

  owner,

  members: [
    {
      user,
      role,
      joinedAt
    }
  ],

  isActive,

  createdAt,
  updatedAt
}
```

APIs Developed

Create Team

POST /api/teams

Purpose:

Create a new team workspace

Security:

ADMIN

SUPER_ADMIN

Get All Teams

GET /api/teams

Purpose:

Fetch all teams

Get Team By ID

GET /api/teams/:id

Purpose:

Fetch a specific team

Update Team

PUT /api/teams/:id

Purpose:

Update team information

Delete Team

DELETE /api/teams/:id

Purpose:

Delete a team workspace

Add Member

POST /api/teams/:id/members

Purpose:

Add user to team

Remove Member

DELETE /api/teams/:id/members/:userId

Purpose:

Remove user from team

Security Implemented

JWT Authentication

Every API protected by:

authenticate

Validation:

Bearer Token

JWT Verification

User Verification

RBAC

Implemented:

SUPER_ADMIN

ADMIN

MANAGER

MEMBER

Example:

Create Team

Delete Team

Allowed:

ADMIN

SUPER_ADMIN

Challenges Faced

Authorization Header Issues

Problem:

Unauthorized

Cause:

Bearer token not passed

Solution:

Authorization: Bearer ACCESS_TOKEN

Team ID vs User ID Confusion

Problem:

Wrong IDs used in APIs

Solution:

Used correct MongoDB ObjectIds

Outcome

Successfully created a production-grade Team Workspace system.

PHASE 3

Meeting Management Module

Objective

Create meeting infrastructure similar to:

Zoom

Google Meet

Microsoft Teams

Meeting Schema

Collection:

meetings

Schema:

```
{  
  title,  
  
  description,  
  
  team,  
  
  host,  
  
  meetingCode,  
  
  scheduledAt,  
  
  startedAt,  
  
  endedAt,  
  
  status,  
  
  participants,  
  
  recordingUrl,  
  
  createdAt,  
  updatedAt  
}
```

Meeting Status Lifecycle

Implemented:

SCHEDULED

↓

ONGOING

↓

COMPLETED

Meeting Code System

Created:

`generateMeetingCode()`

Example:

5536B727

Purpose:

Users join meetings using code
instead of MongoDB IDs

Future:

Meeting Lobby
Socket.IO Rooms
WebRTC Connections

APIs Developed

Create Meeting

POST `/api/meetings`

Purpose:

Create scheduled meeting

Get Meetings

GET `/api/meetings`

Purpose:

Fetch all meetings

Get Meeting By ID

GET `/api/meetings/:id`

Purpose:

Fetch meeting details

Get Meeting By Code

GET /api/meetings/code/:code

Purpose:

Join meetings using meeting code

Update Meeting

PUT /api/meetings/:id

Purpose:

Reschedule or edit meeting

Delete Meeting

DELETE /api/meetings/:id

Purpose:

Remove meeting

Join Meeting

POST /api/meetings/:id/join

Purpose:

Add participant

Leave Meeting

POST /api/meetings/:id/leave

Purpose:

Remove participant

Start Meeting

POST /api/meetings/:id/start

Updates:

status = ONGOING

startedAt = Current Time

End Meeting

POST /api/meetings/:id/end

Updates:

status = COMPLETED

endedAt = Current Time

Security Implemented

Authentication

Protected Routes:

JWT Access Token

Authorization

Meeting Management:

MANAGER

ADMIN

SUPER_ADMIN

Validation

Implemented using:

Zod

Meeting Creation Validation:

```
{  
  title,  
  description,  
  team,  
  scheduledAt  
}
```

Challenges Faced

Team ID vs Meeting ID

Problem:

Used:

`team._id`

instead of:

`meeting._id`

Result:

Meeting not found

Solution:

Used correct meeting id

Invalid URL

Problem:

`http://localhost`

instead of:

`http://localhost`

Solution:

Corrected URL format

Route Ordering Issue

Problem:

`router.get("/:id")`

placed before:

`router.get("/code/:code")`

Result:

Meeting code API failed

Solution:

Specific routes placed above dynamic routes

Duplicate Participant Bug

Problem:

Same participant added multiple times.

Root Cause:

ObjectId comparison mismatch

Solution:

`userId.toString()`

based comparison.

Outcome

Successfully created a production-ready Meeting Management System.

Features available:

Meeting Scheduling

Meeting Lifecycle

Meeting Participation

Meeting Codes

Meeting Lookup

Meeting Management

Current Project Completion

Phase 1

Authentication & RBAC



Phase 2

Team Workspace



Phase 3

Meeting Management



Phase 4

Socket.IO Infrastructure



Phase 5

WebRTC Signaling



Phase 6Real-Time Chat



Phase 7

Notifications



Phase 8

AI Summaries



Phase 9

Task Management & Kanban



Phase 10

Frontend Integration

